COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010-Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	08-11-2026	**DATE SUBMITTED:**	08-11-2026
NAME:	DOLOSO JR, ANDREW S.	**INSTRUCTOR:**	ENGR. JIM LORD QUEJADO

HOA5.1 -

1. PROCEDURE OUTPUT

ILOA

```
#include <iostream>
#include <queue>


void display(std::queue<char> r);
int main(){

    std::queue<char> q;

    q.push('C');
    q.push('P');
    q.push('E');
    q.push('0');
    q.push('1');
    q.push('0');

    //Display
    display(q);


    //empty()
    std::cout<<"is it empty? "<< q.empty()<<std::endl;
    //size()
    std::cout<<"size: "<< q.size()<<std::endl;
```

```
    //front
    std::cout<<"front: "<< q.front()<<std::endl;
    //back
    std::cout<<"back: "<< q.back()<<std::endl;
    //pop
    q.pop();
    //display
    display(q);
    //push another element
    q.push('C');
    q.push('P');
    q.push('E');
    q.push('0');
    q.push('1');
    q.push('0');
    //display
    display(q);
}

void display(std::queue<char> r) { // Automatically copies when called
    while (!r.empty()) {
        std::cout << " " << r.front();
        r.pop();
    }
    std::cout << "\n";
}
```

Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\Documents\CPE009B_Uchi_CPE21S4\HOA5p1>  & 'c:\Users\TIPQC\.vsc
'--stdout=Microsoft-MIEngine-Out-ps55ncvn.5y3' '--stderr=Microsoft-MIEngine-Erro
 C P E 0 1 0
is it empty? 0
size: 6
front: C
back: 0
 P E 0 1 0
 P E 0 1 0 C P E 0 1 0
PS C:\Users\TIPQC\Documents\CPE009B_Uchi_CPE21S4\HOA5p1>
```

Conclusion / Analysis

ILOB

MAIN.CPP

```
#include<iostream>
#include "queuell.h"

int main(){
    qNode<char>* front =  nullptr;
    qNode<char>* back =  nullptr;

    enqueue(&front, &back, 'D');
    std::cout<<front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'R');
    std::cout<<front->data << " "<<back->data<<std::endl;
    enqueue(&front, &back, 'E');
    std::cout<<front->data << " "<<back->data<<std::endl;
     enqueue(&front, &back, 'W');
    std::cout<<front->data << " "<<back->data<<std::endl;
    dequeue(&front, &back);
    std::cout<<front->data << " "<<back->data<<std::endl;
    return 0;
}
```

QUEUELL.H

```
#ifndef QUEUELL_H
#define QUEUELL_H

template <typename T>
class qNode{
    public:
    T data;
```

```
    qNode *next;
};

 //creating anew node
template <typename T>
qNode<T>* new_node(T newData){
qNode<T>* newNode = new qNode<T>;
newNode->data = newData;
newNode->next = nullptr;
return newNode;
}

template<typename T>
void enqueue(qNode<T>** frontPtr, qNode<T>** backPtr, T newData){
    //create new node
    qNode<T>* newNode = new_node(newData);

    //inserting to an empty queue
    if((*frontPtr) == nullptr && (*backPtr)==nullptr){
    (*frontPtr) = newNode;
    (*backPtr) = newNode;
    }

    //inserting an iteminto a non empty queue
    // point the backPtr next to the newNode
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;

}


template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a temporary node to store the node to be deleted
    qNode<T>* deleteNode = nullptr;
    deleteNode = (*frontPtr);
    //check  if the queue is only 1 node
```

```
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
    (*frontPtr) = nullptr;
    (*backPtr) = nullptr;
    delete deleteNode;
    return;
}
    //
    (*frontPtr)= deleteNode->next;
    delete deleteNode;

}

#endif
```

OUTPUT

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\TIPQC\Documents\CPE009B_Uchi_CPE21S4\HOA5p1>  & 'c:\Users\TIPQC\.vscod
'--stdout=Microsoft-MIEngine-Out-kydcefhu.ein' '--stderr=Microsoft-MIEngine-Error-
D D
D R
D E
D W
R W
PS C:\Users\TIPQC\Documents\CPE009B_Uchi_CPE21S4\HOA5p1>
```

DISCUSSION / CODE ANALYSIS

PART 2

main.cpp

```
#include <iostream>
#include "queueArr.h"
#include "queueArr.h"

int main(){



    //create an object
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
    q.Enqueue(2);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
    q.Enqueue(3);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
    q.Enqueue(4);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
     q.Enqueue(5);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
     q.Enqueue(6);
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;

    // dequeue
    q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
    q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
```

```
    q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
    q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
     q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;
     q.Dequeue();
    std::cout<< "q.front: " << q.Front() << " q.back: " << q.Back()
<<"q.size: " << q.Size() << std::endl;

    //copy constructor
    queueArr<int> q2(q);
    std::cout<< "q2.front: " << q2.Front() << " q2.back: " << q2.Back()
<<"q2.size: " << q2.Size() << std::endl;

    //assignment operator
    queueArr<int> q3(6);
    q3 = q;
    std::cout<< "q3.front: " << q3.Front() << " q3.back: " << q3.Back()
<<"q3.size: " << q3.Size() << std::endl;


        return 0;
}
```

queueArr.h

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//array-based Circular Queue
template <typename T>
class queueArr{
```

```
    private:
        size_t q_capacity; //capacity of the array
        size_t q_size; //current size of the array
        size_t q_front; //index of the front item
        size_t q_back; //index of the back item
        T* q_array; //pointer to the array

    public:
        queueArr(size_t capacity); //constructor
        ~queueArr(); //destructor
        bool Empty(); //check if the queue is empty
        bool Full(); //check if the queue is full
        void Clear(); //clear the queue
        T Front(); //return the front item
        T Back(); //return the back item
        size_t Size(); //return the current size of the queue
        void Enqueue(T newDAta); //add an item to the back of the queue
        void Dequeue(); //remove an item from the front of the queue

        // task
        queueArr(const queueArr& other); //copy constructor
        queueArr& operator=(const queueArr& other); //copy assignment
operator

};
//task

//CONSTRUCTOR
template <typename T>
queueArr<T>::queueArr(const queueArr& other) {
    //copy the capacity and size of the other queue
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    //copy the front and back index of the other queue
    q_front = other.q_front;
    q_back = other.q_back;
    //allocate memory for the new array
```

```
    q_array = new T[q_capacity];
    //copy the elements of the other queue to the new array
    for(size_t i = 0; i < q_size; i++){
        size_t index = (q_front + i) % q_capacity;
        q_array[i] = other.q_array[i];
    }
}


//Assignment Opperator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other) {
    //check for self-assignment
    if(this == &other){
        return *this;
    }
    //delete the current array
    delete[] q_array;
    //copy the capacity and size of the other queue
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    //copy the front and back index of the other queue
    q_front = other.q_front;
    q_back = other.q_back;
    //allocate memory for the new array
    q_array = new T[q_capacity];
    //copy the elements of the other queue to the new array
    for(size_t i = 0; i < q_size; i++){
        size_t index = (q_front + i) % q_capacity;
        q_array[i] = other.q_array[i];
    }
    return *this;
}


//constructor:
template <typename T>
```

```
queueArr<T>::queueArr(size_t capacity) {

    // capacity of the array
    q_capacity = capacity;
    // current size of the queue
    q_size = 0;
    // initialize front and back index
    q_front = 0;
    q_back = -1;
    // newly allocated array
    q_array = new T[q_capacity];
    std::cout<< "constructor class was called\n";
}



    template <typename T>
    queueArr<T>::~queueArr() {

        delete[] q_array;
        std::cout<< "Queue Destroyed\n";
    }

    //empty
    template <typename T>
    bool queueArr<T>::Empty(){

        return (q_size == 0);
    }

    //full
    template <typename T>
    bool queueArr<T>::Full(){

        return q_size = q_capacity;
    }

    template <typename T>
```

```
void queueArr<T>::Clear(){

    q_front = 0;
    q_back = -1;
    q_size = 0;
}

template <typename T>
T queueArr<T>::Front(){
    //check and display if the queue is empty
    if(Empty()){
        std::cout<< "Queue is empty\n";
        return T{};
    }
    // return the back item
    return q_array[q_back];
}

template <typename T>
T queueArr<T>::Back(){
    //check and display if the queue is empty
    if(Empty()){
        std::cout<< "Queue is empty\n";
        return T{};
    }
    // return the front item
    return q_array[q_back];
}


template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

template <typename T>
void queueArr<T>::Enqueue(T newData){
```

```
        //check if the queue is full
        if(Full()){
            std::cout<< "Queue is full\n";
            return;
        }
        //increment the rear index
        q_back = (q_back + 1) % q_capacity;
        //add the item to the rear of the queue
        q_array[q_back] = newData;
        //increment the size of the queue
        q_size++;
    }

    template <typename T>
    void queueArr<T>::Dequeue(){
        //check if the queue is empty
        if(Empty()){
            std::cout<< "Queue is empty\n";
            return;
        }
        //increment the front index
        //compute for the q_front to move circular
        q_front = (q_front + 1) % q_capacity;
        //decrement the size of the queue
        q_size--;
    }




    //copy constructor
#endif // QUEUEARR_H
```

DISCUSSION / CODE ANALYSIS

This activity helped me understand how a circular queue works using the FIFO principle. We learned how to add, remove, and process using front, bear, and size. I also learned how C++ classes, constructors, arrays, and dynamic memory can be used to create a practical printer queue system.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

main.cpp

```cpp
#include <iostream>
#include <string>


// create a class called job
class Job{
    private:
        // Id number for the job
        int id;
        // name of the user who submitted the job
        std::string name;
        //number of pages for the printer
        int pages;


    public:
        // constructor
        Job() {
            id = 0;
            name = "";
            pages = 0;
        }


        // constructor for the job class
        Job(int i, std::string n, int p) {
```

```
            id = i;
            name = n;
            pages = p;
        }


        // function to print out the job's details
        void display() {
            std::cout << "Job ID: " << id << ", User: " << name << ",
Pages: " << pages << std::endl;
        }
};


class Printer{
    private:
        const int capacity = 10;
        Job jobs[10];
        int front;
        int rear;
        int count;


    public:
        // create a queue to hold the jobs
        Printer() {
            front = 0;
            rear = -1;
            count = 0;
        }
        // checks if the queue is full
        bool isFull() {
            if (count == capacity) {
                return true;
            }
            return false;
        }
```

```
        // Check if queue is empty
        bool isEmpty() {
            if (count == 0) {
                return true;
            }
            return false;
        }
        // function to add a job to the queue
        void addJob(Job newJob){
            if (isFull()) {
                std::cout << "Printer queue is full. Cannot add new
job." << std::endl;
                return;
            }
            rear = (rear + 1) % capacity; // this moves the rear index
circularly
            jobs[rear] = newJob;
            count++;
            std::cout << "Job added to the printer queue." <<
std::endl;
        }
        // function to process the job from the queue
        void processJob() {
            std::cout << "\nProcessing job:\n" << std::endl;

            while (!isEmpty()) {
                std::cout << "Now printing....." << std::endl;
                jobs[front].display();
                front = (front + 1) % capacity; // this moves the front
index circularly
                count--;
                std::cout << std::endl;
            }
        }
        // shows the waiting line for the jobs awaiting to be processed
        void showQueue() {
            if (isEmpty()) {
```

```
                std::cout << "\nPrinter queue is empty." << std::endl;
                return;
            }


            std::cout << "\nJobs in the printer queue:" << std::endl;
            int index = front;
            for (int i = 0; i < count; i++) {
                jobs[index].display();
                index = (index + 1) % capacity; // this moves the index
circularly
            }
            std::cout << std::endl;
        }
};
int main(){
    //creating of objects for the printer and jobs
    Printer printer;


    int numberOfJobs;
    std::cout << "Enter the number of jobs to add to the printer queue:
";
    std::cin >> numberOfJobs;

    std::cout << std::endl;


    for (int i = 0; i < numberOfJobs; i++) {
        int numPages;
        std::string name;


        std::cout << "Enter User Name: " << (i + 1) << ": ";
        std::cin >> name;
        std::cout << "Enter Number of Pages: " << (i + 1) << ": ";
        std::cin >> numPages;
```

```
        Job newJob(i + 1, name, numPages);
        printer.addJob(newJob);
        std::cout << std::endl;
    }


    // Show the jobs in the queue
    printer.showQueue();

    // Process the jobs in the queue
    printer.processJob();

    // Show the jobs in the queue after processing
    printer.showQueue();


    return 0;
}
```

DISCUSSION / CODE ANALYSIS

This activity helped me understand how a circular queue works using the FIFO principle. We learned how to add, remove, and process using front, back, jobs, and size. I also learned how C++ classes, constructors, arrays, and dynamic memory can be used to create a practical printer queue system.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used:

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.